Parallelizing Single Source Shortest Path (SSSP) on GPU

Olaolu Emmanuel, *Rutgers University*

Abstract

In this project, we are exploring different methods of calculating the single source shortest path by exploiting the large amount of threads available to us by the GPU. To fully take advantage of these threads, we will design our kernels to maximize the amount of active threads at any given point in time. We start by choosing a well known algorithm for calculating the single source shortest path, which lends itself to parallelism, and adding optimizations to further reduce the overall running time.

1. Bellman-ford Algorithm

1.1. Introduction

While this algorithm does not have an amazing runtime, it does have the advantage of being fairly simple to parallelize. This is because each edge is processed independently of the others which makes it easy to distribute the work amongst threads without having to worry about any of the common parallel programming issues such as race conditions.

1.2. Algorithm

The Bellman-ford Algorithm works by keeping two vectors of the distance from the source node to the node at index i. One vector represents the previous distances and the other represents the current. It then iterates through all the edges and updates any distances if it finds a lower one. After one iteration, a check is made to see if any distances have changed and the loop breaks if there have been none.

1.3. Program Design

This program has two main components, a kernel which processes the edges and a host method which continuously calls the kernel until it reaches an iteration where no changes have been made to the distance array. The host method is also responsible for swapping the pointers to the previous and current distance array vectors in between calls to the kernel.

1.4. Execution Configuration

These tests were run using the RoadNetCA sample graph located here: (http://snap.stanford.edu/data/roadNet-CA.html). The GPU used for these computations is the GeForce GT 630.

(256, 8): 37807.326 milli-seconds
(384, 5): 37688.411 milli-seconds
(512, 4): 37893.515 milli-seconds
(768, 2): 38768.479 milli-seconds
(1024, 2): 37465.675 milli-seconds

1.5. Execution Configuration Conclusions

Judging by these tests, it seems that this algorithm performs best with larger block sizes as opposed to a larger number of blocks. This could be due to a block number limit on the GPU that would result in excess blocks having to wait in a queue before they are executed.

1.6. Edge List Organization

These tests were run using the RoadNetCA sample graph located here: (http://snap.stanford.edu/data/roadNet-CA.html). The GPU used for these computations is the GeForce GT 630.

Sorted by source:
(256, 8): 37807.326 milli-seconds
(384, 5): 37688.411 milli-seconds
(512, 4): 37893.515 milli-seconds
(768, 2): 38768.479 milli-seconds
(1024, 2): 37465.675 milli-seconds

Sorted by destination:
(256, 8): 38506.592 milli-seconds
(384, 5): 38421.86 milli-seconds
(512, 4): 38331.245 milli-seconds
(768, 2): 38042.473 milli-seconds
(1024, 2): 38028.632 milli-seconds

1.7. Edge List Organization Conclusions

It seems that the quickest run is still the one which uses the most threads per block. However, something interesting to note is that almost every run of the algorithm sorted by destination is slower than the one sorted by source. This may have to do with the fact that threads in the same block or more likely to be updating the same index since it is sorted by destination. However, this is only an assumption and unfortunately I do not have any statistics to support this claim.

1.8. In-core Configuration

These tests were run using the RoadNetCA sample graph located here: (http://snap.stanford.edu/data/roadNet-CA.html). The GPU used for these computations is the GeForce GT 630.

Sorted by source:
(256, 8): 20248.416 milli-seconds
(384, 5): 20498.864 milli-seconds
(512, 4): 20216.089 milli-seconds
(768, 2): 20777.294 milli-seconds
(1024, 2): 20521.782 milli-seconds

Sorted by destination:
(256, 8): 20780.83 milli-seconds
(384, 5): 20880.516 milli-seconds
(512, 4): 20734.998 milli-seconds
(768, 2): 20813.258 milli-seconds
(1024, 2): 20854.901 milli-seconds

1.9. In-core Configuration Conclusions

It seems that sorting by source yields better runtimes yet again, but I found something else to be interesting in this data. The performance of the in-core version seems to be ~33% better than that of the out-core version. I believe that this is due to memory coalescing since the memory addresses being accessed by the different threads all belong to the same vector which should increase the chance of them being coalesced[1].

1.10. Pros

- This algorithm lends itself to parallelism which makes it easier to translate to the GPU. This is because edges are processed independently of one another which makes distributing work amongst threads much simpler.

1.11. Cons

- There could be some thread divergence when checking to see if an update needs to be made to the distance vector.

2. Work Efficient Algorithm

2.1. Pros

- This algorithm can increase the speed of the edge processing considerably over time since the input amount of edges is reduced every iteration.

2.2. Cons

- There is the added amount of work of having to iterate though the to-process edges and copy them over to the new to-process vector. This can be expensive, especially towards the beginning of the run where almost all edges are in the to-process vector.

3. References

1.https://devblogs.nvidia.com/parallelforall/how-access-global-memory-efficiently-cuda-c-kernels/